DevOps

Course Code: 25CIAEC59/25CYAEC59 | Credits: 1:0:0 | Contact Hours: 15L
Marks: 20 (Component 1: 10 marks, Component 2: 10 marks)

---

1. What This Assignment Is

You will build a small, working DevOps pipeline in two stages:

- Component 1 (10 marks): Put a sample application under version control, build it automatically, run that build through a Continuous Integration (CI) job, and add one CI-related enhancement.
- Component 2 (10 marks): Take that same application, configure a server for it using Ansible, containerize and deploy it on Kubernetes with scaling, and add one deployment-related enhancement.

Together, the two components form one continuous pipeline: code → build → deploy → scale. You do not need two unrelated apps — reuse the same application across both components so the whole thing tells one story.

> ***In simple terms:*** *Think of the whole assignment as one assembly line: you write code, Git stores its history, Jenkins builds and checks it automatically, Ansible sets up a server, Docker packages the app into a portable box, and Kubernetes runs and scales that box. Each phase feeds the next.*

2. Before You Start — Setup Checklist

Install the following on your laptop (or use a cloud/VM environment if your machine can't run Kubernetes locally). Every tool below is free and open-source — no paid plan or paid cloud account is required anywhere in this assignment.

Tool	Purpose	Notes
Git	Version control	git --version to check
JDK 17 + Maven or Gradle	Build the app	Any simple Spring Boot / Node.js / Python app works
Jenkins	CI server	Free, open-source. Run locally or via Docker
Docker Desktop / Docker Engine	Containerize the app	Free for personal/student use
Ansible	Configuration management	Free, open-source. Runs from a Linux/WSL machine
Minikube or Kind	Local Kubernetes cluster	Free, runs entirely on your laptop — no cloud bill
GitHub account (free tier)	Host your repository + optional CI	Free tier covers everything needed here, including GitHub Actions minutes for public repos

3. Component 1 — CI/CD Pipeline with Git, Maven/Gradle & Jenkins (10 marks)

Step 1: Create the Application

Before touching Git, Jenkins, Ansible, or Kubernetes, you need something small to run through the pipeline. Build a simple REST API — a backend service that responds to HTTP requests with data (usually JSON), with no user interface required.

Keep it deliberately simple: 2–3 endpoints, no database (an in-memory list is enough), and something you can build in under an hour. This assignment evaluates how well you set up the pipeline — version control, automated builds, CI, configuration management, and deployment — not how creative or feature-rich the application itself is. A more complex app only adds more ways for something to break, without earning any additional marks.

Example app ideas:

App	What it does
Todo/Task API	Add and list tasks (GET /todos, POST /todos)
Quote Generator API	Returns a random quote from a hardcoded list (GET /quote)
Unit Converter API	Converts values between units (GET /convert?from=km&to=miles&value=5)
Employee Directory API	Lists and adds employee records (GET /employees, POST /employees)
URL Shortener	Shortens a URL and redirects on lookup (POST /shorten, GET /:code)

Use whichever language and framework you're already comfortable with — Java with Spring Boot, Node.js with Express, or Python with Flask/FastAPI are all fine, and each has plenty of beginner tutorials if you get stuck.

Before moving on, confirm the app actually works on its own:

1. Run it locally.
2. Send it a request using curl or Postman.
3. Confirm you get a valid response back.

Only once this works should you move to Step 2 (Git setup) — building the pipeline around a broken app just means debugging two problems at once.

Step 2: Set up Git

1. Initialize a repository and push it to GitHub/GitLab.
2. Create a feature branch (e.g., feature/hello-endpoint), commit your work there.
3. Merge the feature branch into main via a pull request (or git merge).
4. Your repo must show at least 2 branches and a merge in its history — a single commit on main is not enough.

In simple terms: *Git is a tool that remembers every version of your code, so you can go back, compare changes, or work on something new without breaking what already works. A branch is a separate, safe workspace for new changes; a merge combines that work back into the main version once it's ready. This history is what proves you used version control properly, not just uploaded a finished folder.*

Step 3: Configure the Build

- Maven users: fill in pom.xml with your app's dependencies and the Spring Boot (or equivalent) build plugin.
- Gradle users: write build.gradle (Groovy or Kotlin DSL) with the same information.
- Run the build locally first (mvn clean package or ./gradlew build) and confirm it produces a working JAR/artifact before touching Jenkins.

In simple terms: *A build tool turns your source code into something runnable, and manages any libraries your app depends on. Maven/Gradle read a config file listing what your app needs, download those pieces, and package everything into one runnable file. Running this locally first means you catch build problems yourself, instead of discovering them later inside Jenkins where they're harder to debug.*

Step 4: Set up Jenkins (Core — required for full Implementation marks)

1. Create a new Jenkins job (Pipeline or Freestyle — Pipeline is recommended).
2. Under Source Code Management, point it at your Git repository.
3. Add a Build Trigger — either SCM polling (e.g., every minute) or a GitHub webhook.
4. Add build steps that run your Maven/Gradle build command.
5. Add a Post-build Action that archives the build artifact (the JAR/output file).
6. Push a small change to your repo and confirm Jenkins picks it up automatically — a build you have to trigger manually every time does not demonstrate CI.

In simple terms: *Jenkins is a robot that watches your Git repository. Every time you push new code, it automatically downloads it, runs your build command, and tells you (with a green checkmark or a red X) whether it worked — without you having to manually rebuild and test every time yourself. Source Code Management tells Jenkins where your code lives; the Build Trigger tells it when to check; the build step tells it what command to run.*

Step 5: Add ONE CI Enhancement

Choose at least one of the following and implement it for real (not just describe it).

Enhancement	What to do
Automated tests in CI	Add a Test stage in your Jenkinsfile that runs 2–3 unit tests (JUnit for Java, pytest for Python) before the build stage
Image/build versioning	Tag your build artifact or Docker image with the Git commit SHA or Jenkins build number instead of a fixed version
GitHub Actions alongside Jenkins	Add a simple .github/workflows/ci.yml that also builds and tests your app on every push

> ***In simple terms:*** *These are small, realistic additions that real CI pipelines almost always include. A test stage catches broken code before it's built; versioning ties every build back to the exact code that produced it; a second CI tool (GitHub Actions) shows the same automation idea implemented a different way. Pick whichever feels most natural to add to what you've already built.*

What to Capture for Your Report

- Screenshot of your Git branch/commit history (showing the merge)
- Screenshot of the Jenkins job configuration (SCM + trigger + build steps)
- Screenshot of a successful build console output
- Screenshot of the archived artifact
- Screenshot/evidence of whichever CI enhancement you implemented (e.g., test stage passing, versioned artifact name, or Actions workflow run)

4. Component 2 — Configuration Management (Ansible) + Kubernetes Deployment (10 marks)

Use the same application from Component 1.

Step 1: Write the Ansible Playbook

1. Set up an inventory file with at least one target host (can be a local VM or a Docker container acting as a host).
2. Write a playbook that installs the required runtime (e.g., Java), copies your built artifact to the host, and starts the application as a service.
3. Use at least: one variable (e.g., the app port), one handler (e.g., restart the service only when the file changes), and one role (create it with ansible-galaxy init <role-name>).
4. Run it with ansible-playbook -i inventory playbook.yml and confirm it completes without errors.

> ***In simple terms:*** *Ansible configures computers for you using a written recipe (a playbook), instead of you manually typing setup commands on each machine one by one. A variable makes the recipe reusable for different settings (like a port number); a handler runs an action (like a restart) only when something actually changed; a role packages related tasks together so they can be reused across projects. Running the playbook should set up a working server without you touching it by hand.*

Step 2: Containerize the App

Write a Dockerfile that packages your built artifact into an image. Build and run it locally to confirm it works (docker build, docker run) before moving to Kubernetes.

> ***In simple terms:*** *Docker packages your app plus everything it needs to run (like Java or Node.js) into one self-contained image that behaves identically on any computer. A Dockerfile is just the recipe for building that image — start from a base with the right runtime installed, copy in your code, and specify the command that starts the app.*

Step 3: Deploy on Kubernetes

1. Start a local cluster: minikube start (or kind create cluster).
2. Write a Deployment manifest (start with 3 replicas) and a Service manifest (NodePort or ClusterIP) with matching labels/selectors.
3. Apply them: kubectl apply -f deployment.yaml
4. Confirm pods are running: kubectl get pods
5. Access the app through the service and confirm it responds.

In simple terms: *Kubernetes runs your Docker containers for you, keeps them alive, and restarts them automatically if they crash. The Deployment describes what to run and how many copies ("pods") should be up at once. The Service gives those pods one stable address other things can use to reach them, no matter which specific pod actually answers — this is why its selector must match the Deployment's labels exactly.*

Step 4: Demonstrate Scaling

1. Scale up: kubectl scale deployment <name> --replicas=5
2. Run kubectl get pods again and confirm 5 pods are now running.
3. Scale back down and show that too.

In simple terms: *Scaling means changing how many copies of your app are running. Telling Kubernetes to go from 3 to 5 replicas makes it automatically start 2 new pods within seconds — this is the most visually convincing part of the demo, since you can watch new pods appear in real time in the pod list.*

Step 5: Add ONE Deployment Enhancement

Choose at least one of the following and implement it for real.

Enhancement	What to do
Rollback demo	Deploy a working version, intentionally push a broken change, then run kubectl rollout undo deployment/<name> and show the app recover
ConfigMap for settings	Move at least one hardcoded value (like the app port) into a Kubernetes ConfigMap instead of hardcoding it in the YAML
Liveness/readiness probes	Add a livenessProbe and readinessProbe block to your Deployment YAML pointing at your app's root or a /health path
Multi-stage Dockerfile	Split your Dockerfile into a build stage and a slim runtime stage so the final image doesn't carry build tools
Second microservice	Add one more small service (e.g., a second API, or a simple in-memory "database" service) that your main app calls, and deploy both on the cluster
Health/metrics endpoint	Expose a simple /health endpoint (most frameworks provide this for free, e.g., Spring Boot Actuator) and explain in your report what a tool like Prometheus would scrape from it

In simple terms: *A rollback shows Kubernetes can undo a bad deployment automatically — like an undo button. A ConfigMap keeps configuration separate from code, which is standard real-world practice. Probes let Kubernetes check whether your app is actually healthy, not just running. Each option here demonstrates a habit real DevOps engineers use daily — pick whichever pairs most naturally with your app.*

What to Capture for Your Report

- Ansible playbook run output (ansible-playbook command and result)
- Your Dockerfile and confirmation the image builds
- kubectl get pods output before and after scaling
- kubectl get svc output showing the running service
- A screenshot/curl output of the app actually responding
- Screenshot/evidence of whichever deployment enhancement you implemented (e.g., rollback in action, ConfigMap applied, probes configured)

5. What You Must Submit

1. Report — following the provided report template/format: title page and certificate (as given); Abstract, Introduction, Implementation (Development Environment, Code, Description), your chosen Enhancement and why you picked it, Results (screenshots), Conclusion, References — for each component separately.
2. Source code folder — your app code, Jenkinsfile, pom.xml/build.gradle, Ansible playbook + roles, Dockerfile, Kubernetes YAML files, and any files related to your enhancements (test files, workflow YAML, ConfigMap, etc.).
3. A link to your GitHub/GitLab repository (add the course coordinator as a collaborator) — pasted inside the report and also as a text file repo_link.txt.

Do not submit only screenshots without the actual working code — the code will be checked against the screenshots.

6. Resume Framing

- Write one line per component in outcome form, e.g.: "Automated build-and-test pipeline using Jenkins triggered on every Git push; containerized and deployed the service on Kubernetes with autoscaling and rollback support." Your chosen enhancement is exactly what makes this line specific instead of generic.

Swap-In Template (for any app choice)

Fill in the bracketed parts with your own app, language, and enhancements. Everything else can stay as written.

```
[Your App Name] — CI/CD Pipeline & Kubernetes Deployment | [GitHub link]
[Your language/framework], Git, Jenkins, Docker, Ansible, Kubernetes
```

```
• Built a REST API ([1-line description of what it does]) and automated its build
pipeline using Jenkins, triggered on every Git push.
• [Your CI enhancement, e.g., added automated tests / versioned builds by commit SHA /
added GitHub Actions alongside Jenkins].
• Containerized with Docker and deployed on Kubernetes with [N] replicas, using a
Service for internal routing.
• Configured server setup using an Ansible playbook with roles and handlers.
• [Your deployment enhancement, e.g., demonstrated rollback / used ConfigMaps for
configuration / added health checks].
```